

# FRC Java & WPILib

## Programming Onboarding — Curriculum Source Reference

A structured breakdown of Team 2551's video onboarding, prepared as a source document for an AI assistant (Claude Code) to generate website lessons.

|             |                                                                                    |
|-------------|------------------------------------------------------------------------------------|
| Source      | Team 2551 Programming Onboarding video (~1h 26m), 16 chapters                      |
| Stack       | Java · WPILib · command-based robot · VS Code                                      |
| Purpose     | Feed into an AI to scaffold a self-paced lesson website for new programmers        |
| How to read | Each topic gives: Concept → Vocabulary → Verified syntax → Pitfalls → Lesson notes |

### Note on accuracy

The narrator frequently says “I think” or “something like that.” Where the spoken API names differ from real WPILib, this document keeps the teaching intent but adds a **Verified syntax** correction (green boxes) so generated lessons don't inherit mistakes. WPILib changes yearly — always confirm against the current season's docs.

## How to use this document (instructions for the AI)

---

This file is a **source reference**, not a finished lesson set. When asked to build lessons for the tutorial website, treat the content below as the authoritative scope and sequence. Follow these guidelines:

- **One lesson per topic section** unless a topic is large (PID, RobotContainer, Swerve, Vision) — those can be split into 2–3 lessons.
- **Respect the prerequisite order** in the Lesson Progression Map. Do not introduce `PIDController` before `motor.set()`, or commands before the file-structure mental model.
- **Always prefer the green “Verified syntax”** over the narrator’s spoken syntax when writing code samples.
- **Keep the war stories.** They are the most memorable teaching moments (the fried motor, the 2-day voltage bug, coding on the driver station). Turn each into a “Why this matters” sidebar.
- **Audience:** high-school students who can write basic Java and understand objects, methods, and dot-notation. Do not re-teach Java syntax; link to the team’s OOP refresher project instead.
- **Every coding lesson should end with a small, runnable exercise** tied to a real mechanism (intake, flywheel, arm, drivetrain).
- **Mark anything season-specific** (RoboRIO version, banned dashboards, vendordep class names) as “verify for current season.”

### Suggested per-lesson template

#### Lesson title

1. Goal (one sentence: what the student can do after this)
2. Why it matters (use the war story / real mechanism)
3. Concept (plain-language explanation + diagram description)
4. Vocabulary (terms introduced)
5. Code walkthrough (verified syntax, commented line by line)
6. Common mistakes (the pitfalls listed in this doc)
7. Exercise (build/modify a small piece on a real subsystem)
8. Checkpoint questions (3–5 quick self-checks)

## Lesson progression map

Recommended teaching order. The video is organized for watching front-to-back; this regroups it into teachable units with clear prerequisites.

| Unit                     | Lessons (in order)                                                                                         | Prereq     |
|--------------------------|------------------------------------------------------------------------------------------------------------|------------|
| 0 • Orientation          | What a programmer does; mechanics vs. vision vs. autonomous                                                | Basic Java |
| 1 • The Robot's Body     | RoboRIO; Radio; CAN bus; motor controllers (SparkMax / Kraken / Neo)                                       | Unit 0     |
| 2 • The Toolchain        | FRC Game Tools + Driver Station; WPILib + VS Code; REV & CTRE clients; CANm104; dashboards (Glass); Advant | Unit 1     |
| 3 • Project Anatomy      | src tree; the RobotContainer → Command → Subsystem → Motor hierarchy                                       | Unit 2     |
| 4 • Moving a Motor       | Creating a motor object; motor.set(); percent output; intake example                                       | Unit 3     |
| 5 • Sensing              | Encoders: position & velocity; relative vs. absolute                                                       | Unit 4     |
| 6 • Closed-Loop Control  | PID concept (P, I, D); tuning method; PIDController syntax; motion profiling (mentioned)                   | Units 4–5  |
| 7 • Structuring Behavior | Commands (lifecycle); Subsystems (anatomy); Constants                                                      | Units 3–6  |
| 8 • Wiring It Together   | RobotContainer deep dive: bindings, InstantCommand, Sequential/Parallel groups, Robot.java modes           | Unit 7     |
| 9 • Driving              | Swerve concept (field-centric); CTRE generator vs. YAGSL                                                   | Units 6–8  |
| 10 • Seeing              | Limelight & April Tags; TX / TY / ID; vision-aligned PID                                                   | Units 6, 9 |
| 11 • Autonomous          | PathPlanner; paths, named commands, wait commands; auto routines                                           | Units 7–10 |
| 12 • Game Day            | Competition responsibilities; computers & driver station                                                   | All        |

The narrator stresses one dependency repeatedly: **autonomous cannot be built until mechanics and vision both work**. Keep Unit 11 last.

# Unit 1 · The Robot's Body (Hardware & Wiring)

Programmers don't only write software — they also push firmware/updates to hardware, so they must understand how signals travel through the robot.

## 1.1 RoboRIO — the brain

### CONCEPT

- The RoboRIO is the controller that **stores and runs your code**. You “push” compiled code onto it from VS Code.
- Each season you **image** the RIO (flash it with the new firmware) before it will work.
- Season-specific: the video's team used **RoboRIO 1** and expected to move to **RoboRIO 2** next year. *Verify the current controller.*

## 1.2 Radio — communication

### CONCEPT

- A black box that provides the **Wi-Fi link** between the laptop/controller and the robot.
- Connects to the RIO over **Ethernet**. Must be **imaged at competition** (at the field's setup laptop) and **re-imaged at home** afterward.
- After powering on the robot, the radio takes **1–2 minutes** to come online before you can connect.

## 1.3 CAN bus

### CONCEPT

- CAN is the green/yellow twisted-pair wiring carrying **communication** from the RIO to every motor controller.
- It is a **loop / daisy-chain**: every device sits on the same bus and the chain returns to the RIO (passing through the PDH).
- A single broken CAN wire can break the whole loop — *this happens often; expect it.*

## 1.4 Motors & motor controllers

| Device          | What it is                                                               | Which docs          |
|-----------------|--------------------------------------------------------------------------|---------------------|
| <b>SparkMax</b> | Motor controller (the team's main one)                                   | REV / SparkMax docs |
| <b>Neo</b>      | A motor made by REV; plugs into a SparkMax                               | SparkMax docs       |
| <b>Kraken</b>   | A motor (by CTRE) with a built-in controller — CAN runs straight through | CTRE / Kraken docs  |

Lesson note: the key takeaway for students is the **signal path** — RIO → CAN → motor controller → motor → spin — and that the documentation you use depends on which motor you have.

## Unit 2 · The Toolchain (What to Install & Why)

### 2.1 FRC Game Tools & the Driver Station

#### CONCEPT

- **FRC Game Tools** bundles the dashboards and the **Driver Station**.
- The Driver Station is the bar at the bottom of the screen with **Enable/Disable**, mode options, and an output console (your `System.out.println` appears here).
- **Disabled** = no code runs. **Enabled** = controllers active.
- Three status indicators: (a) **Communication** — can it reach the robot; (b) **Robot Code** — is your code present and running; (c) **Joystick** — is a controller connected.

### 2.2 WPILib + VS Code

#### CONCEPT

- Install **WPILib** separately; it installs a dedicated copy of **VS Code**.
- The **WPILib menu** is the small red “W” icon in the top-right of VS Code — this is what makes it more than plain VS Code.
- Two menu actions you’ll use constantly: “**Build/Deploy (Push) Robot Code**” and “**Create New Project.**”
- New projects ask for language (Java) and a **template**: **command-based** (used by the team) or **timed** (simpler but more limiting).

### 2.3 REV & CTRE hardware clients

#### CONCEPT

- **REV Hardware Client**: talk to Neos/SparkMaxes over USB-C. Update firmware, **set CAN IDs**, check sticky faults, and apply a test voltage.
- **CTRE / Phoenix Tuner**: talk to Krakens over the radio. List & **name** motors, set IDs, update firmware.
- **Sticky fault** = a fault that happened at some point (not necessarily now). Clear them once the real problem is fixed.

#### War story — the fried motor (2 lost days)

The team once set a motor’s applied voltage/velocity too high; the motor “spazzed out” and stopped running. They spent two full days hunting the bug before realizing the value was simply too high. Lesson hook: **start low, raise slowly** — and use the hardware client’s “apply voltage” to isolate whether a failure is code, wiring, or build.

### 2.4 CAN IDs

#### CONCEPT

- Because every motor shares one CAN bus, each must get a **unique ID** so code can address it individually (“turn on motor 2”).
- Team convention from the video: **ID 0 is taken by the RoboRIO/system and ID 1 by the PDH**, so **start motor IDs at 2**. (*Treat as a practical convention; confirm reserved IDs for your setup.*)
- The **PDH** (Power Distribution Hub) outputs the red/black power wires and also sits on the CAN chain.

## 2.5 Dashboards & AdvantageScope

### CONCEPT

- **Glass** / SmartDashboard show **real-time values** on screen (e.g., a motor reading 300 RPM). Season note: SmartDashboard was being banned — the team moved to **Glass**. *Verify current dashboard.*
- **AdvantageScope** graphs those values over time (velocity-vs-time, amperage). It's your debugging oscilloscope — e.g., see whether a motor cuts out when it hits an amperage limit.

## Unit 3 · Project Anatomy (The Mental Model)

Before writing anything, students need the file map and the chain of command. This is the single most important conceptual lesson — everything later hangs on it.

### 3.1 The src tree

```
src/ (source – the only folder you usually touch)
  ■■ main/ ... /robot/
    ■■ Robot.java           ← robot lifecycle / modes (mostly leave alone)
    ■■ RobotContainer.java  ← controller bindings + autonomous (MOST important)
    ■■ Constants.java       ← all your tunable numbers in one place
    ■■ subsystems/          ← one file per mechanism; talks to motors
    ■■ commands/            ← tells subsystems what to do; has a defined end
```

Ignore Gradle / build / test files unless you know what they are.

### 3.2 The chain of command

This hierarchy is the backbone of command-based robots:

```
RobotContainer (binds a button press to...)
  ↓
  Command      (a unit of behavior with a clear END; tells...)
  ↓
  Subsystem    (owns the hardware; calls...)
  ↓
  Motor        (finally spins)
```

- **Subsystems** are the only place you actually touch motors.
- **Commands** tell subsystems what to do and, crucially, **have an end** — which lets you sequence behaviors.
- **RobotContainer** wires controller inputs to commands and sets up autonomous.

#### Teaching example — the flywheel

A flywheel has high moment of inertia: hard to start, but stays spinning once moving. A command is perfect here — it can run until the wheel reaches, say, 4000 RPM and only then report `isFinished`. Ramming it to 100%% power instantly would fry the motor (see Unit 4). Use this to motivate why commands need an “end condition.”

## Unit 4 · Moving a Motor (motor.set)

### 4.1 Creating a motor object

#### CONCEPT

Creating a motor is just instantiating a class: a `SparkMax` object needs a **CAN ID** and a **motor type** (brushless/brushed).

#### VERIFIED SYNTAX (REVLIB 2025-STYLE)

```
import com.revrobotics.spark.SparkMax;
import com.revrobotics.spark.SparkLowLevel.MotorType;

// id = the CAN ID you set in the REV Hardware Client
SparkMax motor1 = new SparkMax(2, MotorType.kBrushless);
```

#### Verified syntax note

The narrator wrote `new SparkMax(ID, brushless)` and noted the import “happens automatically.” The real type is `MotorType.kBrushless`. Older code may use the class `CANSparkMax` — confirm which REVLib version your project pulls in.

### 4.2 motor.set() — the most important method

#### CONCEPT

- `motor.set(x)` takes a **percent output** from **-1.0 to 1.0**: 1.0 = 100%% one direction, -1.0 = 100%% the other, 0 = stop.
- You won't know which physical direction is positive until you test — run it slowly and observe.
- Entire simple robots (the cardboard kitbot) have been programmed with **nothing but** `motor.set()`. Use it for systems that just spin until a condition is met.

#### VERIFIED SYNTAX

```
motor1.set(0.7); // run at 70%% power
motor1.set(0.0); // stop
motor1.set(-0.5); // reverse at 50%%
```

#### War story — stay under 80%%

Pushing ~90%% power fried a motor so badly it wouldn't run for more than ~5 seconds. Rule of thumb from the team: **keep output under ~80%%**, or add motors instead of pushing one harder.

Worked example — an intake: a motor drives a pulley that spins roller wheels; `set(0.7)` pulls a game piece in, `set(0)` stops it. Good first hands-on exercise.

## Unit 5 · Sensing (Encoders)

### 5.1 Reading position & velocity

#### CONCEPT

- An **encoder** reports the motor's rotational **position** (an angle, e.g. 42°) and its **velocity**.
- SparkMax and Kraken have **built-in** encoders; you can also clamp a physical encoder onto a shaft.
- Position feeds your PID “current position”; velocity lets you PID-tune a flywheel to a target RPM.

#### VERIFIED SYNTAX

```
double pos = motor1.getEncoder().getPosition(); // angle / rotations
double vel = motor1.getEncoder().getVelocity(); // speed (RPM)
```

### 5.2 Relative vs. absolute

|                   | Relative encoder                     | Absolute encoder                                            |
|-------------------|--------------------------------------|-------------------------------------------------------------|
| <b>On startup</b> | Resets current position to 0         | Remembers last real position (e.g. 50°)                     |
| <b>Good when</b>  | There IS a known/home start position | There is NO physical stop to home against                   |
| <b>Risk</b>       | Loses meaning if you can't home it   | If a motor free-spins after a break, the reference is wrong |

## Unit 6 · Closed-Loop Control (PID)

The video frames PID as “a powerful ally and a dangerous foe.” Students do **not** need calculus — they need to know what each term does and how to tune safely.

### 6.1 What the three terms do

**Error**  $e(t) = |\text{target} - \text{current}|$ . The controller drives error toward zero.

| Term | Name                | Plain-language job                                                                                            | Watch out for                                          |
|------|---------------------|---------------------------------------------------------------------------------------------------------------|--------------------------------------------------------|
| P    | <b>Proportional</b> | Output = $K \times \text{error}$ . As you get closer, push less. Often enough on its own to overcome friction | Too small near the end to overcome friction            |
| I    | <b>Integral</b>     | Slowly builds added power over time to overcome friction/gravity                                              | Over-shoots & oscillation if tuned too high            |
| D    | <b>Derivative</b>   | Uses the rate of change $\times K$ to damp the approach so you don't fly right                                | Tighten but faster oscillation; can leave small offset |

### 6.2 The tuning method (teach this as a recipe)

- Start with **only KP**, very small. Raise it slowly (the narrator used steps of ~5; smaller is fine).
- If it **oscillates** or won't settle, adjust **KD**.
- If it **misses the target by a little** and you can afford correction time, add a small **KI**.
- **Change only one variable at a time** — otherwise you can't tell what helped.
- Be patient. Every season the team has broken something (bent parts, a whole mechanism falling off) from bad PID.

### 6.3 Verified syntax

```
import edu.wpi.first.math.controller.PIDController;

// PIDController takes three doubles: kP, kI, kD
PIDController pid = new PIDController(0.05, 0.0, 0.0);

double current = motor1.getEncoder().getPosition();
double output = pid.calculate(current, 50.0); // 2nd arg = setpoint
// (setpoint optional; defaults to 0)

motor1.set(output); // feed the controller output straight into the motor
```

#### Verified syntax note

The narrator named the controller variable `e` and called `e.calculate(current, target)` — correct. He also mentioned **motion profiling** as an alternative the team skipped because the syntax changed between seasons. Treat motion profiling as an optional advanced lesson.

# Unit 7 · Structuring Behavior (Commands, Subsystems, Constants)

## 7.1 Command lifecycle

### CONCEPT

A command has four lifecycle pieces. The narrator's spoken names are simplified; the verified WPILib method names are in the code box.

| Narrator said | Real method                     | Runs...                                |
|---------------|---------------------------------|----------------------------------------|
| start         | <b>initialize()</b>             | once, when the command begins          |
| periodic      | <b>execute()</b>                | repeatedly, every ~20 ms, while active |
| end           | <b>end(boolean interrupted)</b> | once, when the command stops           |
| isFinished    | <b>isFinished()</b>             | every cycle; return true to end        |

### Verified syntax note

`isFinished()` returning **true** ends the command immediately; returning **false** means it never ends on its own. `execute()` (the narrator's "periodic") and the subsystem's own `periodic()` both run on the ~20 ms loop — don't confuse the two.

## 7.2 Subsystem anatomy

```
public class IntakeSubsystem extends SubsystemBase {
    // 1) declare hardware/variables
    private final SparkMax roller = new SparkMax(2, MotorType.kBrushless);

    // 2) constructor initializes things
    public IntakeSubsystem() { /* setup */ }

    // 3) your own methods — the ONLY place motors get touched
    public void runRoller(double speed) { roller.set(speed); }
    public void stop() { roller.set(0); }

    // 4) periodic() runs every ~20 ms
    @Override
    public void periodic() {
        // e.g. System.out.println(roller.getEncoder().getPosition());
    }
}
```

## 7.3 Constants

- **Constants.java** is just a home for your tunable numbers (speeds, IDs, setpoints).

- Other files read them the normal Java way, so changing the intake speed means editing **one line** in one place.
- WPILib's "Create New Project" scaffolds the Constants file for you — copy its pattern per subsystem.

## Unit 8 · Wiring It Together (RobotContainer & Robot.java)

### 8.1 RobotContainer — the most important file

- **Top:** declare subsystems and commands, e.g. `IntakeCommand x = new IntakeCommand(intakeSub);`
- **Constructor:** set up dashboard widgets and an optional **autonomous picker**, then call your `configureBindings()` method.
- **Bindings:** map controller buttons to commands.

#### VERIFIED SYNTAX — BUTTON BINDINGS

```
import edu.wpi.first.wpilibj2.command.button.CommandPS5Controller;
import edu.wpi.first.wpilibj2.command.InstantCommand;

CommandPS5Controller controller = new CommandPS5Controller(0);

// PS5 face buttons: cross(), circle(), square(), triangle()
controller.cross().onTrue(intakeCommand);
controller.cross().onFalse(stopCommand);
controller.square().whileTrue(intakeCommand); // runs only while held

// No full command? Wrap a subsystem call in an InstantCommand (runs once):
controller.triangle().onTrue(
    new InstantCommand(() -> intakeSub.runRoller(0.7))
);
```

#### Verified syntax note

The narrator used `.x()` for “the X button.” On a **PS5** controller the cross button method is `.cross()` (an **Xbox** controller uses `.a()`, `.b()`, ...). The `() -> ...` inside `InstantCommand` is a **lambda**; students can treat it as “run this line when pressed.”

### Grouping commands

```
// Run in order, each waits for the previous to finish:
new SequentialCommandGroup(cmd1, cmd2, cmd3);

// Run all at the same time:
new ParallelCommandGroup(cmdA, cmdB);
```

### 8.2 Robot.java — modes

- Four modes: **teleoperated**, **autonomous**, test, practice. You pick the mode in the Driver Station. Focus on teleop & autonomous; test/practice are optional.
- Each mode has an **Init** (runs once at mode start) and a **Periodic** (runs every ~20 ms): `teleopInit` / `teleopPeriodic`, `autonomousInit` / `autonomousPeriodic`, plus `robotInit` / `robotPeriodic`.

- You usually leave this file alone.

#### Verified syntax note

The narrator garbled init vs. periodic (“init is what happens when it stops”). Correct it in lessons: **Init runs once when the mode starts; Periodic runs every ~20 ms while in that mode.** The team did edit `testInit` once to run a PathPlanner routine for testing — a good “when you’d actually touch Robot.java” example.

## Unit 9 · Driving (Swerve)

### 9.1 Field-centric control — the key idea

#### CONCEPT

- Swerve is **field-centric**, not game-like. Pushing the left stick “forward” moves the robot toward the **far side of the field, regardless of which way the robot is facing**.
- Whatever direction you push on the stick is the direction the robot translates on the field (think top-down map view).
- The **right stick rotates** the robot left/right without changing its translation direction.

### 9.2 You don’t hand-write swerve math

- Swerve involves heavy trig, so teams use a **code generator**. Each swerve module has **two motors**: one for **drive**, one for **rotation** (so 4 modules = 8 motors).
- **CTRE Swerve Generator** (Phoenix Tuner): fast and easy, but requires **8 Kraken/CTRE motors**. You feed it robot dimensions and center of mass; it writes the code to import.
- **YAGSL** (Yet Another Generic Swerve Library): works across motor brands, but harder to set up.

Lesson note: students rarely write swerve from scratch — the lesson goal is understanding field-centric behavior and knowing which generator matches the team’s hardware.

## Unit 10 · Seeing (Limelight & April Tags)

### 10.1 The hardware

- A **Limelight** is a camera (by Citrus Circuits) that detects **April Tags** — QR-code-like fiducial markers placed around the field.
- Configure it at `limelight.local` (brightness, etc.). **Re-tune before every competition** for the venue lighting.

### 10.2 Syntax & the key readings

- Limelight provides a helper **file** (`LimelightHelpers`), not a class you instantiate. Common methods: `getTX`, `getTY`, `getFiducialID`.
- Best practice from the video: wrap these in your own **Limelight subsystem** with readable method names.

| Reading   | Geometry                                    | What you do with it                                             |
|-----------|---------------------------------------------|-----------------------------------------------------------------|
| <b>TX</b> | Horizontal angle to the tag (overhead view) | PID-tune swerve so $TX \rightarrow 0$ to face/aim at the target |
| <b>TY</b> | Vertical angle to the tag (side view)       | Changes with distance $\rightarrow$ PID your distance/position  |
| <b>ID</b> | Which April Tag you see                     | Choose which tags to act on or ignore                           |

This unit is where PID (Unit 6) and swerve (Unit 9) pay off: vision gives the error, PID drives it to zero. The team used TX to aim at a scoring goal and TY to range a target.

## Unit 11 · Autonomous (PathPlanner)

Autonomous is the first ~20 seconds of a match where the robot runs itself. Build it **last** — it reuses the commands you already wrote for mechanisms.

### 11.1 PathPlanner workflow

- **PathPlanner** (a Windows app on the driver station) shows a visual field. You place a start pose and draw the robot's route.
- It uses robot **constants** (weight, moment of inertia, dimensions) to compute the motor power and swerve commands to follow your route — approximately.
- Because it's not perfect, pair it with **alignment / position-correction code** (your PID + vision).

### 11.2 Building a routine

- **Paths**: point A → point B segments.
- **Named commands**: a special command type you register (e.g. "lowerIntake") so the auto can call your mechanism behaviors.
- **Wait commands**: pause N seconds before continuing.
- Assemble paths + named commands + waits into an **auto routine**; expose it through the autonomous picker in RobotContainer.

#### Dependency reminder

The narrator is emphatic: you **cannot** build a reliable autonomous until mechanics and vision are solid, because auto is literally a sequence of those commands. Keep this unit last in the course.

## Unit 12 · Game Day (Competition & Computers)

### 12.1 Your competition responsibilities

- **Image the radio** at the field's setup laptop before you play.
- Go to the field with the drive team and bring the **driver station laptop**. Plug in three things: **Ethernet** (to the field), the **controller** (USB), and **power**.
- Your job is **not** to coach the driver (that's the drive coach). It's to **watch the console for errors** — e.g. "CAN ID 16 timed out" signals a wrong ID or (more often) a wiring break — and answer the driver's questions.

### 12.2 Computers

- The team has Macs, a PC, and the Windows **driver-station laptop**. **Macs can't run the Driver Station**, so use the dedicated laptop for that.
- In a pinch you can even write code directly on the driver-station laptop.

### War story — coding on the driver station

The captain's personal computer broke halfway through the season, so he did his coding on the driver-station laptop for the rest of the year. Good “the show goes on” note for new members who worry about not having a powerful machine.

## Appendix A · Master Syntax Cheat-Sheet

Verified WPILib-style syntax, consolidated for quick reference when generating code samples. Always confirm class names against the current season's vendordeps.

```
// ---- MOTOR -----
SparkMax motor = new SparkMax(2, MotorType.kBrushless);
motor.set(0.7);           // percent output, -1.0 .. 1.0
motor.set(0);             // stop

// ---- ENCODER -----
double pos = motor.getEncoder().getPosition();
double vel = motor.getEncoder().getVelocity();

// ---- PID -----
PIDController pid = new PIDController(kP, kI, kD);
double out = pid.calculate(currentPosition, targetPosition);
motor.set(out);

// ---- COMMAND LIFECYCLE -----
initialize()           // once at start
execute()              // every ~20 ms while running
end(boolean interrupted) // once when stopping
isFinished()          // return true to end

// ---- BUTTON BINDINGS (RobotContainer) -----
CommandPS5Controller c = new CommandPS5Controller(0);
c.cross().onTrue(cmd);
c.cross().onFalse(cmd);
c.square().whileTrue(cmd);
c.triangle().onTrue(new InstantCommand(() -> sub.method()));

// ---- COMMAND GROUPS -----
new SequentialCommandGroup(cmd1, cmd2, cmd3); // in order
new ParallelCommandGroup(cmdA, cmdB);         // together

// ---- ROBOT MODES (Robot.java) -----
robotInit / robotPeriodic
teleopInit / teleopPeriodic
autonomousInit / autonomousPeriodic

// ---- OUTPUT -----
System.out.println(value); // shows in Driver Station console
```

## Appendix B · Glossary

|                |                                                                          |
|----------------|--------------------------------------------------------------------------|
| <b>RoboRIO</b> | The controller that stores and runs your code; imaged each season.       |
| <b>Radio</b>   | Wi-Fi link between laptop/controller and robot; imaged at comp and home. |
| <b>CAN bus</b> | Looped comms wiring connecting the RIO to every motor controller.        |

|                                     |                                                                                  |
|-------------------------------------|----------------------------------------------------------------------------------|
| <b>PDH</b>                          | Power Distribution Hub; sends power out and sits on the CAN chain.               |
| <b>SparkMax</b>                     | REV motor controller (drives Neo motors).                                        |
| <b>Neo</b>                          | REV motor; plugs into a SparkMax.                                                |
| <b>Kraken</b>                       | CTRE motor with a built-in controller; CAN runs straight through it.             |
| <b>Sticky fault</b>                 | A fault that occurred at some point (not necessarily current); clear once fixed. |
| <b>WPILib</b>                       | The core library + custom VS Code for FRC robot code.                            |
| <b>Driver Station</b>               | App with Enable/Disable, mode select, status lights, and console output.         |
| <b>Glass / SmartDashboard</b>       | Real-time value dashboards (SmartDashboard being phased out).                    |
| <b>AdvantageScope</b>               | Tool that graphs robot values over time for debugging.                           |
| <b>Subsystem</b>                    | File that owns a mechanism's hardware; the only place motors are touched.        |
| <b>Command</b>                      | A behavior with a defined end; tells subsystems what to do.                      |
| <b>RobotContainer</b>               | Wires controller inputs to commands; holds autonomous setup.                     |
| <b>Constants</b>                    | Central file of tunable numbers.                                                 |
| <b>PID</b>                          | Proportional-Integral-Derivative closed-loop control toward a target.            |
| <b>Encoder</b>                      | Sensor giving a motor's position and velocity (relative or absolute).            |
| <b>Swerve</b>                       | Field-centric drivetrain; each module has a drive + a rotation motor.            |
| <b>CTRE Generator / YAGSL</b>       | Tools that generate swerve code (CTRE = Krakens; YAGSL = generic).               |
| <b>Limelight</b>                    | Vision camera that detects April Tags.                                           |
| <b>April Tag</b>                    | Fiducial marker (QR-like) used to locate the robot on the field.                 |
| <b>TX / TY / ID</b>                 | Limelight readings: horizontal angle, vertical angle, tag identity.              |
| <b>PathPlanner</b>                  | App for drawing and generating autonomous routines.                              |
| <b>Named command / Wait command</b> | PathPlanner building blocks: a registered behavior; a timed pause.               |

## Appendix C · External documentation to cite in lessons

- **WPILib docs** — the authoritative reference for everything above; verify each season.
- **REV / SparkMax docs** — for Neo + SparkMax APIs.
- **CTRE / Phoenix (Kraken) docs** — for Kraken motors and the swerve generator.
- **Limelight docs** — the “holy grail” for vision setup and methods.
- **PathPlanner docs** — for autonomous routines.
- Team’s internal **OOP refresher project** (on the Google Drive) — prerequisite for new programmers.

Reminder for the AI: the video itself notes WPILib changes yearly. Flag any class names, RoboRIO versions, or dashboard choices as “verify for the current season” in generated lessons.